

Python, Pandas, NumPy & Calculus

Interview Prep Notes

1. Difference between is and ==

`==` checks whether two values are equal. `is` checks whether two variables refer to the same object in memory.

```
a = [1, 2, 3]
b = [1, 2, 3]

a == b    # True
a is b    # False
```

Interview point: Use `==` for value comparison and `is` mainly for identity checks such as `x is None`.

2. Error handling in Python

Python handles errors using `try`, `except`, `else`, and `finally`.

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Done")
```

- **try** → code that may cause an error
- **except** → handles the error
- **else** → runs if no error occurs
- **finally** → runs regardless of whether an error occurs

You can also raise your own errors using **raise**.

3. NumPy indexing

NumPy allows you to access elements of arrays using indexes.

```
import numpy as np

arr = np.array([10, 20, 30, 40])

arr[0]      # 10
arr[1:3]    # [20 30]
arr[-1]     # 40
```

For 2D arrays:

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
```

```
arr[0, 1]    # 2
arr[:, 1]    # [2 5]
arr[1, :]    # [4 5 6]
```

Important concepts: integer indexing, slicing, boolean indexing, fancy indexing, and multidimensional indexing.

4. Pandas indexing

Pandas mainly provides two important indexing methods:

.loc[] → label-based indexing

```
df.loc[2, "Name"]
df.loc[0:3, ["Name", "Age"]]
```

.iloc[] → position-based indexing

```
df.iloc[2, 1]
df.iloc[0:3, 0:2]
```

Easy way to remember: *loc = labels* | *iloc = integer positions*

5. Pandas filtering

Filtering means selecting rows that satisfy a condition.

```
df[df["Age"] > 25]
```

Multiple conditions:

```
df[(df["Age"] > 25) & (df["City"] == "Kochi")]
```

Common operators:

- `>` `<` `>=` `<=`
- `==` `!=`
- `&` → AND
- `|` → OR
- `~` → NOT
- `.isin()` → check multiple values
- `.str.contains()` → filter text

6. Advanced data manipulation with Pandas

This is a large topic. Important areas include:

- `groupby()`

- `agg()`
- `merge()`
- `join()`
- `concat()`
- `pivot_table()`
- `melt()`
- `apply()`
- `transform()`
- `map()`
- sorting
- handling missing values
- duplicate handling
- reshaping data
- MultiIndex

Example:

```
df.groupby("Department")["Salary"].mean()
```

This groups employees by department and calculates the average salary for each department.

7. Limits

A limit describes the value that a function approaches as the input approaches a particular value.

Although the expression is undefined at $x = 1$, as x gets closer to 1, the function approaches 2.

$$\text{We write: } \lim_{x \rightarrow 1} (x^2 - 1) / (x - 1) = 2$$

Important concepts:

- Left-hand limit
- Right-hand limit
- Two-sided limit
- Limits at infinity
- Continuity
- Indeterminate forms such as $0/0$

8. Derivatives

A derivative tells us the rate at which something is changing. Geometrically, it represents the slope of a curve at a particular point.

$$m = (y_2 - y_1) / (x_2 - x_1)$$

For example:

$$f(x) = x^2$$

Then:

$$f'(x) = 2x$$

So at $x = 3$, the slope is 6.

Applications include:

- Finding slopes
- Optimization
- Velocity/acceleration
- Machine learning
- Understanding how variables change

9. Higher-order derivatives

A higher-order derivative means taking the derivative more than once.

$$\text{For: } f(x) = x^3$$

First derivative:

$$f'(x) = 3x^2$$

Second derivative:

$$f''(x) = 6x$$

Third derivative:

$$f'''(x) = 6$$

The second derivative is particularly useful for understanding curvature, concavity, and optimization.

10. Intuition behind Gradient Descent

Gradient descent is an optimization algorithm used to minimize a function, especially a loss/cost function in machine learning.

Imagine you're standing on a mountain and want to reach the lowest point.

- The gradient tells you which direction is uphill.
- So you move in the opposite direction.
- You repeat this until you reach a minimum.

The basic update is:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla J(\theta)$$

where:

- θ = model parameters

- α = learning rate
- $\nabla J(\theta)$ = gradient of the loss function

Simple interview answer: "Gradient descent is an optimization algorithm that iteratively updates model parameters in the opposite direction of the gradient of the loss function, so that the loss decreases toward a minimum."

Priority for Studying

I'd prioritize them like this:

High priority:

Pandas indexing/filtering → Advanced Pandas → Derivatives → Gradient Descent

Medium priority:

NumPy indexing → Limits → Higher-order derivatives → Error handling

Quick topic:

is vs ==